JS

Курс из 3 частей · для тех, кто уже знает C# и Python

JavaScript для конкурса

Часть 1. Основы языка и карточки на странице

ЧАСТЬ 1 · СЕЙЧАС

Основы JS и карточки из массива

ЧАСТЬ 2

JSON, поиск, фильтр, сортировка, кнопки

ЧАСТЬ 3

Удаление, localStorage, финальный проект

ТЕОРИЯ · 1

Подключаем JS и открываем консоль

КОД · index.html + script.js

```
<!-- index.html, внутри <head> -->
<script src="script.js" defer></script>

// script.js
console.log("Привет, JS!");
console.log(2 + 2, "товаров:", 5);
let shop = "Гаджет-шоп";
console.log("Магазин:", shop);
```

ЧТО ПРОИЗОЙДЁТ

Открой страницу, нажми **F12** → вкладка **Console**:

```
▸ Console
Привет, JS!
4 'товаров:' 5
Магазин: Гаджет-шоп
```

defer = «запусти скрипт, когда весь HTML загрузится». Без него JS может не найти элементы страницы.

console.log печатает через запятую сколько угодно значений. Твой главный друг при отладке.

Как это в C# / Python: console.log(...) = Console.WriteLine(...) = print(...). Только вывод ищи не в терминале, а в F12.

ТЕОРИЯ · 2

JS глазами C# и Python: только отличия

Что	C#	Python	JavaScript
Переменная	`int n = 5;`	`n = 5`	`let n = 5;`
Константа	`const int N = 5;`	`N = 5` (просто имя)	`const N = 5;`
Типы	строгие	динамические	динамические, как в Python
Функция	`int Sum(int a, int b)`	`def sum(a, b):`	`function sum(a, b) { }`
Условие	`if (x > 0) { }`	`if x > 0:`	`if (x > 0) { }` — как в C#
Цикл по списку	`foreach (var p in list)`	`for p in list:`	`for (const p of list)`
Цикл с i	`for (int i=0; i<n; i++)`	`for i in range(n):`	`for (let i = 0; i < n; i++)`
Равенство	`a == b`	`a == b`	`a === b` ← три знака!
И / ИЛИ / НЕ	`&& || !`	`and or not`	`&& || !`
«Ничего»	`null`	`None`	`null` и `undefined`

Как это в C# / Python: скобки { } и точка с запятой — как в C#, а тип переменной не пишешь — как в Python.

⚠ ЛОВУШКИ JS · 1–3

Три ошибки, на которых спотыкаются все

⚠ Ловушка JS

== и ===

```
5 == "5" // true 😱
5 === "5" // false ✅
0 == "" // true 😱😱
```

== перед сравнением «подгоняет» типы друг к другу. Пиши **всегда === и !==** — они работают как привычное == в C#.

⚠ Ловушка JS

undefined и null

```
const p = { title: "Мышь" };
p.price // undefined
let pick = null; // пусто
```

undefined — «значение не задали». Опечатка в имени поля тоже даёт undefined, и **без ошибки! null** — ты сам сказал «пусто».

⚠ Ловушка JS

let и const

```
const price = 100;
price = 200; // ❌ TypeError
const list = [];
list.push(1); // ✅ можно
```

const нельзя переприсвоить, но менять содержимое массива можно. По умолчанию const, **let** — если значение меняется. **var** — забудь.

Как это в C# / Python: в C# == и так строгое; null ≈ None, а вот undefined нет ни там, ни там — это «фишка» JS.

ТЕОРИЯ · 3

Функции, стрелка => и строки с ${ }

КОД · script.js

```
// 1) Обычная функция — как def / метод
function priceText(price) {
  return price + " ₽";
}

// 2) То же самое, но стрелочной функцией
const priceText2 = (price) ⇒ price + " ₽";

// 3) Шаблонная строка: `текст ${значение}`
const title = "Мышь";
console.log(`${title}: ${priceText(1490)}`);
console.log(priceText2(2990));
```

ЧТО ПРОИЗОЙДЁТ

▸ Console

```
Мышь: 1490 ₽
2990 ₽
```

Стрелка по кусочкам:

(price) — что функция получает на вход
=> — читай «превращается в»
price + " ₽" — что вернуть (return не нужен)

Обратные кавычки ` (клавиша Ё) делают шаблонную строку: всё внутри **${ }** JS вычислит и вставит в текст.

Как это в C# / Python: лямбда price => price + " ₽" в C# — та же стрелка! В Python: lambda и f"{title}: ..."

ТЕОРИЯ · 4

Объект — один товар из каталога

КОД · script.js

```
// Объект = набор «ключ: значение»
const product = {
  id: 1,
  title: "Наушники",
  price: 2990,
};

console.log(product.title); // читаем
product.price = 2490; // меняем
product.inStock = true; // добавляем
console.log(product);
```

ЧТО ПРОИЗОЙДЁТ

▸ Console

```
Наушники
▸ {id: 1, title: 'Наушники',
   price: 2490, inStock: true}
```

• Поле читаем через точку: **product.title**. Можно и как словарь: **product["title"]**.

• Присвоил несуществующему полю — оно **появилось**. Прочитал несуществующее — получил **undefined**.

• Объект в const, а поля меняются — помнишь ловушку №3?

Как это в C# / Python: это dict из Python с доступом через точку, или объект класса из C#, но без класса.

ТЕОРИЯ · 5

Массив объектов — весь каталог

КОД · script.js

```
// Массив объектов = наш каталог
const products = [
  { id: 1, title: "Наушники", price: 2990 },
  { id: 2, title: "Мышь", price: 1490 },
  { id: 3, title: "Клавиатура", price: 3990 },
];
console.log(products.length); // длина
console.log(products[0].title); // первый
products.push({ id: 4, title: "Колонка", price: 2490 });
for (const p of products) {
  console.log(p.title, p.price);
}
```

ЧТО ПРОИЗОЙДЁТ

▸ Console

```
3
Наушники
Наушники 2990
Мышь 1490
Клавиатура 3990
Колонка 2490
```

- **products[0]** — первый товар, **products[0].title** — его название. Индексы с 0.
- **push** добавил 4-й товар в конец, поэтому цикл вывел четыре строки.

Как это в C# / Python: List<T>.Add / append → **push**, .Count / len() → **length**, foreach / for in → **for...of**.

ТЕОРИЯ · 6

JS меняет страницу: найти, создать, положить

КОД · script.js

```
// в HTML: <h1 id="title">Магазин</h1>
// <div id="list"></div>

// 1) Найти (как в CSS: # — id, . — класс)
const title = document.querySelector("#title");
title.textContent = "Гаджет-шоп 🎮";

// 2) Создать новый тег и положить внутрь
const list = document.querySelector("#list");
const item = document.createElement("p");
item.textContent = "Мышь — 1490 ₽";
list.appendChild(item);
```

ЧТО ПРОИЗОЙДЁТ

ДО

Магазин

#list пустой

ПОСЛЕ

Гаджет-шоп 🎮

Мышь — 1490 ₽

querySelector — найти элемент
textContent — прочитать или заменить текст
createElement — создать тег (пока «в воздухе»)
appendChild — положить внутрь → он появился

Как это в C# / Python: как label.Text = "..." в WinForms или label.config(text="...") в tkinter.

ПРАКТИКА · ШАГ 1 ИЗ 2

createCard: один товар → одна карточка

Подготовка: в index.html пустой <main id="catalog"></main>, в script.js — массив products.

КОД · script.js

```
// Получает объект — отдаёт HTML-элемент
function createCard(p) {
  const card = document.createElement("div");
  card.className = "card"; // класс из CSS
  card.innerHTML = `
    <h3>${p.title}</h3>
    <p>${p.price} ₽</p>
  `;
  return card; // отдаём готовую карточку
}

console.log(createCard(products[1]));
```

ЧТО ПРОИЗОЙДЁТ

```
▸ Console
▾ <div class="card">
    <h3>Мышь</h3>
    <p>1490 ₽</p>
  </div>
```

Мышь
1490 ₽

Так она будет выглядеть, но на странице её **пока нет** — положим на шаге 2.

innerHTML читает строку как HTML-теги, **textContent** — как простой текст.

Как это в C# / Python: метод, который делает и возвращает новый объект; HTML внутри — как f-строка с вставками.

ПРАКТИКА · ШАГ 2 ИЗ 2

render(list): данные → страница

КОД · script.js

```
const catalog = document.querySelector("#catalog");

// Рисует ВЕСЬ переданный список заново
function render(list) {
  catalog.innerHTML = ""; // 1) очистить
  for (const p of list) { // 2) каждый товар
    catalog.appendChild(createCard(p)); // 3)
  }
}

render(products); // запуск!
```

ЧТО ПРОИЗОЙДЁТ

ДО #catalog пустой

ПОСЛЕ

Наушники 2990 ₽	**Мышь** 1490 ₽
Клавиатура 3990 ₽	**Колонка** 2490 ₽

⚠ Ловушка JS: innerHTML += в цикле

Каждый += заново пересобирает весь каталог: медленно и ломает кнопки внутри. Клади карточки через appendChild.

Как это в C# / Python: foreach + panel.Controls.Add(card) или for p in items: frame.append(card).

МИНИ-ПРОЕКТ

Твой каталог: своя тема, 5 карточек

Задание

1. Выбери тему: книги, игры, еда, кроссовки — что угодно.
2. index.html: заголовок и пустой <main id="catalog">.
3. script.js: массив из 5 объектов, у каждого id, название и ещё минимум 2 поля.
4. Функция createCard(item) — карточка с 3+ полями.
5. Функция render(list) — выводит все 5 карточек.
6. style.css: сетка карточек и класс .card.
7. Бонус: счётчик «Всего: 5» и значок через if (например, «Хит», если рейтинг выше 4.7).

Чек-лист самопроверки

- ☑ В консоли F12 нет красных ошибок
- ☑ Только const и let, никакого var
- ☑ Все сравнения через ===
- ☑ В HTML нет карточек — их рисует JS
- ☑ Поменял цену в массиве — поменялась карточка
- ☑ Добавил 6-й объект через push — появилась 6-я карточка
- ☑ Нигде не видно слова undefined

Решение для сравнения: папка lesson1-solution — «Книжная полка». Сначала сделай сам, потом сравни!

ПРОВЕРЬ СЕБЯ

Три задачки без подсказок

1

Что выведет консоль?

```
console.log(3 == "3");
console.log(3 === "3");
console.log(products[5]);
```

В products четыре товара, как на слайде с render.

2

Найди две ошибки

```
const count = 0;
count = count + 1;
const p = { title: "Мышь" };
console.log(p.titel);
```

Одна ошибка красная в консоли, другая — тихая.

3

Перепиши короче

```
function hello(name) {
  return "Привет, " + name;
}
```

Сделай стрелочную функцию в одну строку и используй ${ }.

Ответы (не подглядывай!): 1) true, false, undefined — шестого товара нет. 2) count должен быть let; titel — опечатка, тихий undefined. 3) const hello = (name) => `Привет, ${name}`;

ИТОГИ ЧАСТИ 1

Что ты теперь умеешь

- ✓ Подключать script.js и читать консоль F12
- ✓ Писать let/const, ===, стрелки => и `${}`
- ✓ Хранить каталог в массиве объектов
- ✓ Находить элементы и менять их текст
- ✓ Делать карточки: createElement + appendChild
- ✓ Рисовать весь каталог одной функцией render(list)

ДАЛЬШЕ · ЧАСТЬ 2

Данные оживают

Массив переедет в **data.json** и будет загружаться через fetch. Добавим кнопки-фильтры, поиск прямо при вводе и сортировку по цене.

Твоя **render(list)** станет главной героиней: изменили список → вызвали render.